Information Technologies for Industrial Engineers

JavaScript

- JavaScript is the Programming Language for the Web
- Can be used on server-side
  - `Node.js`
- Stack Overflow Survey 2025

TypeScript

- TypeScript is a syntactic superset of JavaScript which adds static typing.
- It shares the same base syntax as JavaScript, but adds something to it.
- *Much beter than JavaScript, trust me.*

Initializing a variable

```
let myName = "Me";
const yourName = "You";
```

Variable types

```
const numberType = 1;
const stringType = "This is a string";
const booleanType = true;
const undefinedType = undefined;
const nullType = null;
const arrayType = [1, 2, 3];
const objectType = { key1: 1, key2: "two" };
```

To check type

```
console.log(typeof numberType);
```

Function

```
// Function
function hello1(name: string) {
  console.log("Hello " + name);
}

// Arrow function
const hello2 = (name: string) => {
  console.log("Hello " + name);
};

// Calling
hello1("Ari");
hello2("Tom");
```

`if` - `else if` - `else`

```
let choice = "A"; // 'A', 'B', 'C'

if (choice === "A") {
  console.log("You chose A.");
} else if (choice === "B") {
  console.log("You chose B.");
} else if (choice === "C") {
  console.log("You chose C.");
} else {
  console.log("You did not choose A, B, or C.");
}
```

Ternary Operator `? :`

- A compact shorthand for a simple `if` / `else`
- Syntax: `condition ? valueIfTrue : valueIfFalse`

```
const age = 20;

// if / else version
let status;
if (age >= 18) {
  status = "adult";
} else {
  status = "minor";
}

// Ternary version (same result)
const status2 = age >= 18 ? "adult" : "minor";
console.log(status2); // "adult"
```

Ternary Operator — More Examples

```
const score = 75;

// Inline in a template literal
console.log(`Result: ${score >= 50 ? "Pass" : "Fail"}`); // "Result: Pass"

// Nested (use sparingly — can hurt readability)
const grade = score >= 80 ? "A" : score >= 70 ? "B" : score >= 60 ? "C" : "F";

console.log(grade); // "B"
```

> For simple true/false values, prefer the ternary operator over `if` / `else`.

Falsy Values

- A **falsy** value is treated as `false` in a boolean context (e.g., inside `if`)
- All other values are **truthy**

Falsy Values

Value	Type
`false`	Boolean
`0`	Number
`""` or `''`	String (empty)
`null`	Null
`undefined`	Undefined
`NaN`	Not-a-Number

Falsy Values — Example

```
const values = [false, 0, "", null, undefined, NaN, "hello", 42, []];

for (const v of values) {
  if (v) {
    console.log(`${String(v)} is TRUTHY`);
  } else {
    console.log(`${String(v)} is FALSY`);
  }
}
```

Falsy Values — Example

- Useful for checking if a variable has a meaningful value:

```
let name = "";

if (!name) {
  console.log("Name is not set"); // runs because "" is falsy
}
```

Logical OR `||`

- Returns the **first truthy** value, or the last value if none are truthy
- Often used to provide a **default value**

```
const input = "";
const name = input || "Anonymous";
console.log(name); // "Anonymous"  (because "" is falsy)

const score = 0;
const display = score || 100;
console.log(display); // 100  ⚠️ 0 is falsy, so this may be unexpected!
```

Nullish Coalescing `??`

- Returns the **right side** only when the left is `null` or `undefined`
- **Ignores** other falsy values like `0`, `""`, `false`

```
const score = 0;
const display = score ?? 100;
console.log(display); // 0  ✅ 0 is kept because it is not null/undefined

const name = null;
const displayName = name ?? "Anonymous";
console.log(displayName); // "Anonymous"
```

> Use `??` when `0`, `""`, or `false` are valid values you want to keep.

Logical AND `&&`

- Returns the **first falsy** value, or the last value if all are truthy
- Often used for **conditional execution**

```
const isLoggedIn = true;
const username = "Ari";

// Only evaluate the right side if the left is truthy
isLoggedIn && console.log(`Welcome, ${username}!`); // prints

const isAdmin = false;
isAdmin && console.log("Admin panel"); // does NOT print
```

`||`, `??`, `&&` — Summary

Operator	Returns	Use case
`||`	First **truthy** value	Default when value may be falsy
`??`	Right side only if left is `null` / `undefined`	Default without overriding `0` or `""`
`&&`	First **falsy** value (or last if all truthy)	Conditional execution

`||`, `??`, `&&` — Summary

```
let a = null;
console.log(a || "fallback"); // "fallback"
console.log(a ?? "fallback"); // "fallback"

let b = 0;
console.log(b || "fallback"); // "fallback"  ⚠️
console.log(b ?? "fallback"); // 0           ✅
```

Looping code

- Print 10 random numbers

```
for (let i = 0; i < 10; i++) {
  console.log(Math.random()); // 10 randoms numbers
}
```

Looping through an array

```
const cats = ["Leopard", "Serval", "Jaguar", "Tiger", "Caracal", "Lion"];

for (const cat of cats) {
  console.log(cat);
}
```

Template Literals

- Use backticks `` ` `` instead of quotes
- Embed expressions with `${}`

```
const name = "Ari";
const age = 25;

// Old way
console.log("My name is " + name + " and I am " + age + " years old.");

// Template literal
console.log(`My name is ${name} and I am ${age} years old.`);

// Multi-line string
const message = `Hello, ${name}!
Welcome to the class.`;
console.log(message);
```

Switch Statement

```
let day = "Monday";

switch (day) {
  case "Monday":
    console.log("Start of the work week.");
    break;
  case "Friday":
    console.log("End of the work week!");
    break;
  case "Saturday":
  case "Sunday":
    console.log("Weekend!");
    break;
  default:
    console.log("Midweek day.");
}
```

Objects

- A collection of key-value pairs

```
const student = {
  name: "Ari",
  age: 20,
  major: "Industrial Engineering",
};

// Accessing properties
console.log(student.name); // dot notation
console.log(student["age"]); // bracket notation

// Modifying properties
student.age = 21;

// Adding new properties
student.gpa = 3.8;
```

Destructuring

- Extract values from arrays or objects into variables

```
// Array destructuring
const colors = ["red", "green", "blue"];
const [first, second] = colors;
console.log(first); // "red"

// Object destructuring
const student = { name: "Ari", age: 20, major: "IE" };
const { name, age } = student;
console.log(name); // "Ari"
console.log(age); // 20

// Rename while destructuring
const { name: studentName } = student;
console.log(studentName); // "Ari"
```

Array Methods (1/2)

- `length` — get the number of elements
- `forEach` — execute a function for each element
- `map` — transform each element
- `filter` — keep elements that pass a test
- `find` — get the first matching element
- `findIndex` — get the index of the first matching element

Array Methods (2/2)

- `splice` — add/remove elements at a specific index
- `push` — add to the end
- `pop` — remove from the end
- `includes` — check if an element exists
- `join` — combine elements into a string

Array Methods Examples (1/3)

```
const numbers = [1, 2, 3, 4, 5];

// length: get the number of elements
console.log(numbers.length); // 5

// forEach: execute a function for each element
numbers.forEach((n) => console.log(n)); // prints 1, 2, 3, 4, 5

// map: double each number
const doubled = numbers.map((n) => n * 2);
console.log(doubled); // [2, 4, 6, 8, 10]

// filter: keep only even numbers
const evens = numbers.filter((n) => n % 2 === 0);
console.log(evens); // [2, 4]
```

Array Methods Examples (2/3)

```
// find: first number greater than 3
const found = numbers.find((n) => n > 3);
console.log(found); // 4

// findIndex: index of first number greater than 3
const index = numbers.findIndex((n) => n > 3);
console.log(index); // 3

// splice: remove and/or add elements
const removed = numbers.splice(1, 1, 99); // remove 1 element at index 1, add 99
console.log(numbers); // [1, 99, 3, 4, 5]
console.log(removed); // [2]

const fruits = ["apple", "banana", "cherry"];
```

Array Methods Examples (3/3)

```
// push: add to end
fruits.push("mango");
console.log(fruits); // ["apple", "banana", "cherry", "mango"]

// pop: remove from end
const last = fruits.pop();
console.log(fruits); // ["apple", "banana", "cherry"]

// includes: check if an element exists
console.log(fruits.includes("banana")); // true

// join: combine elements into a string
console.log(fruits.join(", ")); // "apple, banana, cherry"
```

TypeScript: Type Annotations

- Explicitly declare the type of a variable

```
let score: number = 95;
let username: string = "Ari";
let isActive: boolean = true;
let scores: number[] = [90, 85, 78];
```

- TypeScript will show an error if types don't match

```
let score: number = "high"; // ❌ Error: Type 'string' is not assignable to type 'number'
```

TypeScript: Function Types

```typescript
// Parameter types and return type
function add(a: number, b: number): number {
  return a + b;
}

// Arrow function with types
const greet = (name: string): string => {
  return `Hello, ${name}!`;
};

// Optional parameter (?)
function introduce(name: string, age?: number): string {
  if (age !== undefined) {
    return `I'm ${name}, ${age} years old.`;
  }
  return `I'm ${name}.`;
}
```

TypeScript: Interface

- Define the shape of an object

```
interface Student {
  name: string;
  age: number;
  major: string;
  gpa?: number; // optional property
}

const s1: Student = {
  name: "Ari",
  age: 20,
  major: "Industrial Engineering",
};

const s2: Student = {
  name: "Tom",
  age: 22,
  major: "Mechanical Engineering",
```

TypeScript: Type Alias

- Create a reusable name for a type

```
type ID = number | string; // union type

let userId: ID = 101;
userId = "abc-999"; // also valid

type Point = {
  x: number;
  y: number;
};

const origin: Point = { x: 0, y: 0 };
```

Spread Operator `...`

- Expand arrays or objects

```
const a = [1, 2, 3];
const b = [4, 5, 6];
const combined = [...a, ...b];
console.log(combined); // [1, 2, 3, 4, 5, 6]

const defaults = { theme: "dark", language: "en" };
const userSettings = { language: "th", fontSize: 14 };

// Later properties override earlier ones
const settings = { ...defaults, ...userSettings };
console.log(settings);
// { theme: "dark", language: "th", fontSize: 14 }
```

Optional Chaining `?.`

- Safely access properties or call methods on objects that might be `null` or `undefined`
- Returns `undefined` instead of throwing an error

Optional Chaining `?.`

```
const user = {
  name: "Ari",
  address: {
    city: "Bangkok",
  },
};

// Without optional chaining (risky!)
// console.log(user.profile.age); // ❌ Error if profile is undefined

// With optional chaining (safe!)
console.log(user.profile?.age); // undefined (no error)
console.log(user.address?.city); // "Bangkok"
```

Optional Chaining — More Examples

```
const users = [
  { name: "Ari", email: "ari@example.com" },
  { name: "Tom" }, // no email
];

// Optional property access
console.log(users[0].email?.toLowerCase()); // "ari@example.com"
console.log(users[1].email?.toLowerCase()); // undefined (no error)

// Optional method call
const greet = (user) => user.sayHello?.(); // calls if method exists

// Optional array index
const firstItem = users?.[0]; // undefined if users is null/undefined
```